



北京邮电大学
Beijing University of Posts and Telecommunications

移动终端安全

课程研究报告

题目	移动应用逆向分析与防篡改技术研究
姓名	
学号	
班级	
指导教师	

移动应用逆向分析与防篡改技术研究

李昊伦

北京邮电大学

{lhldudu}@bupt.edu.cn

摘要：移动智能终端已经广泛进入学习、工作和生活场景，其开放系统、无线联网和第三方应用安装等特性，在带来便利的同时也放大了安全风险。Android 等移动系统虽然通过进程沙箱、应用签名、权限控制、SELinux 和可信执行环境等机制保护系统和用户数据，但由于应用来源复杂、设备型号差异较大，仍容易受到恶意软件、隐私窃取、应用重打包和运行时注入等威胁。本文以“移动应用逆向分析与防篡改技术”为主题，介绍 APK/HAP 包结构、字节码反编译、Smali 修改、应用签名、代码混淆、加固保护、反调试、反 Hook 和完整性校验等内容，并结合 crackme 类样例的破解和注册机分析过程整理移动应用的攻防思路。最后，本文讨论了 HarmonyOS、OpenHarmony 等国产系统在应用安全和权限管理方面的发展情况，分析其在统一生态、签名校验、AccessToken 权限模型、应用市场审核和软硬件协同等方面的优势，并指出其在生态成熟度、第三方安全工具链、跨端攻击面治理和开发者安全能力方面仍有不足。

关键词：移动应用安全；Android；逆向分析；应用加固；防篡改；HarmonyOS；OpenHarmony

一、移动应用逆向分析与防篡改技术概述

1.1 研究背景与发展历程

移动智能终端已经不再只是早期用于通话和短信的通信工具，而是逐渐发展为集社交、支付、办公、出行、医疗、娱乐和身份认证等功能于一体的个人数字中枢^[1]。开放式操作系统和第三方应用安装能力，是智能手机区别于传统功能机的重要特征。用户可以根据需要安装各类应用，开发者也能够基于操作系统提供的接口开发复杂业务功能，从而推动移动终端形成了规模庞大、功能丰富的应用生态。

从技术演进看，早期移动终端系统较为封闭，第三方应用能力有限，攻击面主要集中在短信、彩信、基带、蓝牙、浏览器和少量平台接口。Android 兴起之后，移动应用安全问题明显扩大。一方面，Android 基于 Linux 内核，采用应用沙箱、UID 隔离、权限控制和签名机制，为应用生态提供了基础安全模型；另一方面，Android 的开放性使 APK 获取、解包、反编译、重打包和第三方分发都变得相对容易。相关教材对 Android 系统安全机制与对抗技术的讨论也说明，高市场占有率和开放生态

会引发木马病毒、隐私窃取、偷跑流量、恶意扣费、应用审核不足等问题^[1]。随着移动支付、短视频、电商、游戏和金融应用成为日常生活的入口，攻击者对应用内部逻辑的兴趣也从“好奇式破解”转向了有组织的黑产牟利。

移动应用逆向分析的发展大致可以分为四个阶段。第一阶段是静态解包和资源修改阶段。由于 APK 本质上是 ZIP 归档，攻击者可以直接查看 Manifest、资源文件、证书信息和 DEX 字节码，进行图标替换、广告注入或简单逻辑修改。第二阶段是字节码反编译阶段。随着 ApkTool、dex2jar、JADX、JD-GUI、Baksmali/Smali 等工具成熟，攻击者可以把 DEX 转换为可读性较高的 Java/Kotlin 代码或 Smali 汇编，从而定位注册验证、支付校验、授权判断、加密函数和网络接口。第三阶段是动态调试与运行时 Hook 阶段。Frida、Xposed、Magisk、LLDB、IDA、Ghidra 等工具使攻击者能够在应用运行时劫持函数、修改返回值、Dump 内存、绕过证书绑定和反调试逻辑。第四阶段是自动化和产业化阶段。黑产组织会把静态分析、脚本化 Patch、批量重签名、自动化投放、流量变现、账号盗取和数据倒卖连接起来，使逆向分析不再只是单点技术，而成为攻击链条中的基础能力。OWASP Mobile Top 10 也长期将反编译、二进制保护不足和逆向篡改列为移动应用的重要风险来源^[6]。

表 1: Android 应用逆向分析常用工具链对比

工具/方法	主要对象	典型用途	局限性
ApkTool	APK、资源、Smali	解包、资源还原、Smali 修改、重打包	Java 语义不如 JADX 直观；加固壳下只能看到壳代码
JADX/JD-GUI	DEX/JAR	快速查看 Java/Kotlin 近似源码，定位业务逻辑与字符串	混淆后可读性下降；反编译结果可能与真实控制流不完全一致
Baksmali/Smali	Dalvik 字节码	精确修改条件跳转、返回值、常量和方法调用	阅读门槛较高，容易因寄存器和方法签名错误导致崩溃
Frida/Xposed	运行时函数	Hook 校验函数、加解密函数、网络函数，观察动态数据	容易被反 Hook、完整性检测和环境检测发现
IDA/Ghidra	Native so 库	分析 C/C++ 核心算法、JNI、反调试和加固壳逻辑	分析成本高，对符号缺失和混淆二进制不友好
apksigner	APK 签名块	对重打包 APK 重新签名，或验证签名方案和证书信息 ^[4]	只能说明签名是否有效，不能证明业务逻辑可信

防护技术的发展同样经历了由简单到复杂的过程。早期应用主要依靠 ProGuard 混淆、签名校验和少量关键字符串加密。后来 Android Gradle 插件内置 R8，混淆、压缩和优化成为常规发布流程。再之后，第三方加固服务开始普及，通过 DEX 加密、壳加载、反调试、反 Hook、JNI 下沉、控制流混淆、资源加密、完整性校验等方式提高逆向成本。近年来，移动安全领域又把防护重点扩展到供应链安全、应用市场审核、软件物料清单、隐私合规、动态风险检测、可信执行环境、密钥安全和服务端风

控。也就是说，移动应用安全已经从“应用包内部保护”扩展成了“终端、应用、系统、分发渠道和服务端共同参与”的安全工程。

1.2 基本概念与技术对象

移动应用逆向分析是指在没有完整源代码或不依赖源代码的情况下，对应用安装包、运行时行为、网络通信、内存数据和系统交互进行分析，以还原程序结构、业务逻辑、数据流和安全机制。逆向分析本身并不天然等于攻击，它在漏洞研究、恶意样本分析、兼容性调试、取证和安全评估中都有正当用途。但当逆向结果被用于绕过授权、破解付费功能、植入恶意代码、窃取用户数据或仿冒官方应用时，就会构成严重安全风险。

与之对应，防篡改技术是指检测、阻止或提高应用被非授权修改的成本。它关注的不是单纯“别人能不能看到代码”，而是“别人看到后能不能低成本地修改并重新分发”“运行时能不能被 Hook 或调试”“关键业务结果能不能被伪造”“用户和系统能不能识别应用是否来自可信开发者”。因此，防篡改技术一般包含代码混淆、签名校验、完整性校验、反调试、反 Hook、运行环境检测、加固壳、白盒加密、证书绑定、服务端校验和风险监控等多个层面。

从内容边界看，移动应用逆向分析与防篡改至少包含四类核心问题。第一类是“应用包和代码表示”，即 APK/HAP 的结构、DEX/ArkTS/Native 二进制的表达方式、资源文件和签名块的组织方式；第二类是“分析技术”，包括静态反编译、符号搜索、控制流追踪、动态调试、Hook、抓包和内存 Dump；第三类是“篡改方式”，包括 Smali Patch、资源替换、Manifest 修改、恶意 SDK 注入、重新签名和仿冒分发；第四类是“防护与治理”，包括代码混淆、签名校验、完整性检测、反调试、反 Hook、服务端风控、应用市场审核和供应链治理。这样划分的理由是，攻击者从“看懂程序”到“修改程序”再到“分发获利”需要经过多个环节，防护也必须在这些环节上形成对应控制。

APK 是 Android 应用安全分析的核心对象。一个典型 APK 中包含 AndroidManifest.xml、classes.dex、resources.arsc、res 资源目录、lib 原生库目录、META-INF 签名相关文件等内容。AndroidManifest.xml 描述应用包名、组件、权限、入口 Activity、服务、广播接收器、ContentProvider 等；classes.dex 保存 Java/Kotlin 编译后的 Dalvik/ART 字节码；lib 目录保存 ARM、ARM64、x86 等架构下的 native so 库；META-INF 或 APK Signing Block 则承担签名和完整性校验相关功能。攻击者通常先看 Manifest 判断入口和权限，再看 DEX 查找关键逻辑，如果存在 native 层逻辑，再转向 IDA、Ghidra 等工具分析 ELF 文件。

字节码和 Smali 是 Android 逆向中最常见的分析对象。Java/Kotlin 代码经过编译后不会直接变成机器码，而是先生成 class，再转换为 DEX。DEX 中仍保留较多类名、

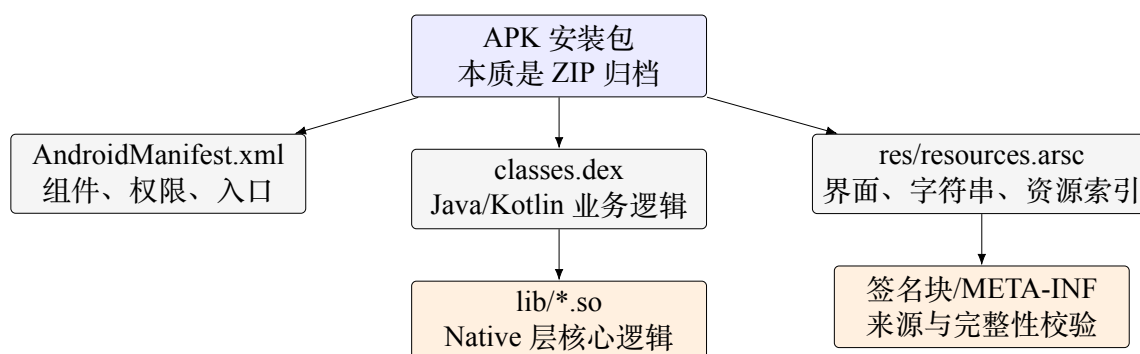
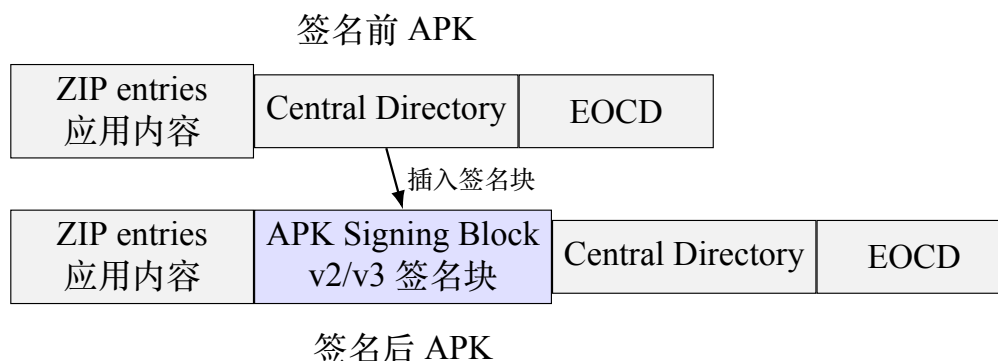


图 1: APK 文件结构与逆向关注点

方法名、调用关系和字符串信息，因此可读性通常高于原生机器码。Smali 是 DEX 的一种汇编表示，接近虚拟机指令层。对 crackme 类程序而言，攻击者常常不需要完全理解所有业务逻辑，只要定位判断分支，例如“if-eqz”“if-nez”或某个返回布尔值的函数，就能通过修改跳转、强制返回 true、替换比较常量等方式绕过验证。

应用签名是移动应用身份和完整性的基础机制。Android 要求安装的应用必须签名。官方 AOSP 文档显示，APK Signature Scheme v2 是 whole-file signature scheme，会在 ZIP Central Directory 前插入 APK Signing Block，并保护 APK 主要内容的完整性^[2]；v3 在 v2 基础上加入 supported SDK versions 和 proof-of-rotation，用于支持签名密钥轮换^[3]。Android 9 及以上系统可以按 v3、v2 或 v1 验证 APK，验证过程包括定位签名块、选择签名算法、验证 signed data、计算 APK 内容摘要并与签名块中的 digest 比对等。Android SDK Build Tools 中的 apksigner 也明确提示：如果 APK 签名后又被修改，签名会失效^[4]。签名机制的重要意义在于：如果攻击者修改 APK 后重新打包，原签名会失效；系统在升级同包名应用时也会检查签名是否匹配，防止不同开发者随意替换已有应用。但签名不是万能的。攻击者可以对篡改后的 APK 重新自签名并通过非官方渠道诱导用户安装，因此应用签名必须和应用市场审核、用户来源识别、运行时自校验、服务端风控共同使用。

图 2: APK Signature Scheme v2 签名块位置示意（根据 Android 官方文档^[2] 整理）

OWASP MASVS 将移动应用安全控制划分为存储、密码、认证、网络、平台、代

码、抗逆向与防篡改、隐私等多个控制组，其中 MASVS-RESILIENCE 明确关注对逆向分析和篡改尝试的抵抗能力，并把反篡改、反静态分析、反动态分析等作为韧性控制点^[5]。这个分类对课程报告很有启发：移动应用防护不应只盯着“混淆是否打开”，而要把代码、平台交互、网络、认证、隐私和运行时环境统一考虑。例如，一个金融应用即使代码混淆做得很好，如果登录态 token 明文存储在外部存储，或者服务端完全相信客户端的“支付成功”标志，那么整体仍然是不安全的。

1.3 典型攻击路径

移动应用逆向攻击通常不是一步完成，而是沿着“获取样本、静态分析、动态验证、修改重打包、绕过检测、分发牟利”的路径推进。

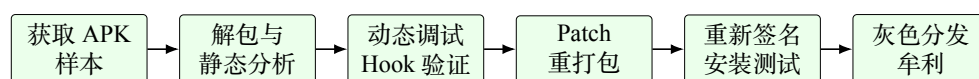


图 3: 移动应用逆向篡改的一般流程

第一步是样本获取。攻击者可以从应用市场、第三方下载站、设备本地备份、抓包缓存或灰色渠道获得 APK。Android 应用包较容易被复制，因此攻击者可以在离线环境中反复分析，不必直接攻击服务端。对于 HAP、APP Pack 等国产系统应用包，虽然格式和工具链不同，但“安装包作为离线分析对象”这一风险仍然存在。

第二步是静态分析。攻击者解包 APK，读取 Manifest 中的权限和组件，分析资源文件中的 URL、接口路径、错误提示、密钥片段、渠道号、广告 SDK 信息等。随后使用 JADX 或 dex2jar 查看 Java 层代码，搜索关键词，如 register、license、vip、pay、check、verify、token、serial、md5、aes、rsa、debug、emulator 等。很多 crackme 或低安全强度应用的关键点都可以通过字符串搜索定位。

第三步是动态分析。静态混淆或加固会降低代码可读性，但应用运行时必须执行真实逻辑。攻击者可以用模拟器、真机、Frida、Xposed、LLDB、Android Studio Profiler、日志和抓包工具观察行为。典型目标包括 Hook 掉签名校验函数、替换授权函数返回值、拦截加解密函数输入输出、绕过 SSL Pinning、Dump 内存中的 DEX 或密钥、伪造设备信息和网络响应。

第四步是修改和重打包。静态 Patch 常见方式包括修改 Smali 分支、替换常量、删除校验调用、改变资源配置、插入恶意代码、添加广告 SDK、修改 Manifest 权限和组件导出状态。修改完成后，攻击者用 ApkTool 重打包，再用自己的证书签名。对于没有严格来源校验和服务端联动的应用，这种重打包版本可能仍然能运行。

第五步是传播和变现。攻击者可能通过第三方应用市场、社交群、论坛、网盘、短链接、钓鱼网站等方式传播篡改版应用。变现方式包括盗取账号、窃取隐私、植入

广告、诱导充值、盗刷支付、恶意扣费、挖矿、代理流量和供应链攻击。对开发者而言，这不仅是收入损失，也是品牌信誉和用户安全的损失。

1.4 典型防护机制

代码混淆是最基础的防护。R8/ProGuard 可以重命名类、方法和字段，移除无用代码，压缩资源，并在一定程度上优化控制流。混淆的优点是成本低、对开发流程影响小、能显著降低初级逆向效率。缺点也很明显：混淆不是加密，字符串、调用关系、异常信息、资源名称、JNI 接口和网络行为仍可能暴露；而且过度混淆可能影响反射、序列化、路由框架和第三方 SDK，因此需要编写 keep 规则。

字符串加密和资源加密用于保护 URL、密钥片段、错误提示和配置资源。它们可以避免攻击者通过简单搜索定位关键逻辑。但如果解密函数在客户端执行，攻击者仍可以 Hook 解密函数或在运行时 Dump 明文。因此，字符串加密应与运行时检测、服务端校验和密钥分段结合，而不能把长期密钥完整放在客户端。

签名校验和完整性校验用于发现重打包。应用可以在运行时读取自身签名证书摘要、计算 DEX 或 so 文件哈希，与预置值或服务端返回值比对。如果发现签名不匹配、文件被修改、加载路径异常或安装来源异常，则拒绝运行或降低权限。这里的难点是校验逻辑本身也可能被 Hook 或 Patch，因此关键校验最好多点部署、Java/native 双层实现，并与服务端风控联动。

反调试和反 Hook 用于提高动态分析成本。常见反调试方法包括检测 `Debug.isDebuggerConnected`、检查 `/proc/self/status` 中 `TracerPid`、检测 `ptrace` 状态、扫描调试端口、检测可疑进程和库名。反 Hook 方法包括检测 Frida server、扫描内存中 `frida/xposed/substrate` 字符串、检查函数入口是否被修改、验证 so 段权限、检测类加载器异常等。它们的局限是对抗性很强：攻击者可以改名、隐藏、Patch 检测函数或在更底层绕过。因此反调试和反 Hook 更适合作为“提高成本和触发风险评分”的手段，而不应成为唯一防线。

DEX 加固和壳保护是商业应用常见方案。壳程序会把原始 DEX 加密或压缩，应用启动时由加载器在内存中解密并加载真实代码。这样静态分析时只能看到壳逻辑，攻击者需要先脱壳。现代加固还会叠加虚拟机保护、控制流混淆、JNI 下沉、反 Dump、反模拟器、反注入和完整性校验。加固的优势是见效快，适合保护商业逻辑、游戏逻辑和金融风控逻辑；不足是可能引入兼容性、性能、崩溃排查和合规问题。如果开发者把所有安全希望都寄托在加固厂商上，而业务协议和服务端校验很薄弱，加固被绕过后仍会暴露核心风险。

白盒密码和密钥保护用于在不可信客户端环境中降低密钥被直接提取的概率。移动端经常需要保存接口签名密钥、离线授权密钥或内容解密密钥。普通 AES/RSA

实现中，密钥最终会以某种形式出现在内存中。白盒密码试图把算法和密钥混合在查表、编码和混淆结构中，使攻击者难以恢复原始密钥。但白盒密码也不是绝对安全，更适合保护中短期业务密钥，并配合密钥轮换、TEE、服务端验证和异常检测。

服务端风控是移动应用防篡改中最容易被低估的一层。客户端永远处于用户控制的环境，不能被完全信任。注册、支付、积分、会员、游戏道具、优惠券、风控评分等关键状态必须由服务端最终判定。客户端可以做体验层面的预校验，但不能让“客户端返回 true”成为授权依据。服务端应记录设备指纹、签名证书摘要、安装来源、版本、行为频率、接口调用链路和异常 Hook 信号，对疑似篡改客户端降权或封禁。

表 2: 移动应用防篡改措施的效果与局限

防护措施	主要防护目标	优点	局限与注意点
R8/ProGuard 混淆	降低 Java/Kotlin 代码可读性	成本低，适合默认开启；能减少符号和无用代码	不能保护运行时数据；反射和序列化需要 keep 规则
字符串/资源加密	保护 URL、提示语、配置和密钥片段	可降低关键词搜索和快速定位效率	解密逻辑仍在客户端，可被 Hook 或 Dump
签名与完整性校验	识别重打包、文件修改、来源异常	与系统安装校验和服务端校验结合后效果较好 ^{[2][4]}	校验函数本身可能被 Patch，需要多点和服务端联动
反调试/反 Hook	发现 Frida、Xposed、调试器等动态分析环境	能提高动态攻击成本，适合风控评分	误报和绕过都较常见，不宜作为唯一阻断条件
DEX 加固/壳保护	阻碍静态反编译和批量 Patch	对商业应用见效快，可保护核心逻辑入口	可能影响兼容性、性能和崩溃排查；仍可能被脱壳
服务端风控	防止篡改客户端获得真实业务收益	能控制支付、会员、积分等最终状态	需要日志、模型和应急策略，开发成本较高

1.5 相关研究与写作依据

在梳理移动应用逆向分析与防篡改技术的同时，本文也参考了移动应用安全领域的一些典型研究，以便从更完整的角度分析移动终端安全问题。Enck 等人在 USENIX Security 2011 对 1100 个 Android 免费应用进行了安全研究，指出应用权限、隐私数据流和第三方组件会共同影响终端安全^[14]。这一研究说明，移动应用安全不能只看某个单独函数是否安全，而要同时关注权限、数据、组件和网络行为，因此本文把逆向分析放在移动终端整体安全模型中讨论。

重打包风险是本文选择“防篡改”作为重点的直接原因。Zhou 等人提出的 Droid-MOSS 面向第三方 Android 应用市场检测重打包应用，其基本思路是利用代码指纹和相似性分析识别被修改后重新发布的应用^[15]。这说明 APK 容易被复制、修改和再次分发不是理论风险，而是应用市场长期存在的现实问题。因此，本文在攻击路径

中专门分析“修改、重签名、灰色分发”这一链条，并在防护部分强调签名校验、完整性校验和应用市场治理。

混淆、控制流保护和反调试技术是本文讨论“提高逆向成本”的依据。Android 代码保护综述认为，混淆、字符串加密、动态加载、native 下沉等技术能够增加静态分析成本，但它们通常只能提高攻击门槛，不能彻底消除逆向可能性^[16]。针对 Dalvik 字节码的控制流混淆研究也表明，控制流变换可以降低反编译代码的可读性，但仍需要与数据保护、运行时检测和服务端验证配合使用^[17]。此外，大规模 Android 应用研究发现，反调试和反篡改保护在真实应用中的部署并不充分，且很多应用只使用单一检测手段^[18]。这些研究共同支持本文的一个判断：防篡改应采用分层策略，而不是依赖某一个加固工具或某一条检测规则。

表 3: 相关研究对本文分析框架的支撑

研究方向	代表性研究	对本文的启示
Android 应用安全整体研究	Enck 等对 Android 应用权限和安全行为的研究 ^[14]	分析移动应用安全时，需要同时考虑权限、数据流、组件暴露和网络行为
重打包检测	DroidMOSS 使用代码指纹识别重打包应用 ^[15]	防篡改不仅是本地代码保护，也需要关注应用市场和分发渠道治理
代码混淆与保护	Android 代码保护综述和控制流混淆研究 ^{[16][17]}	混淆能提高逆向成本，但不能替代签名、完整性校验和服务端授权
反调试与反篡改部署	大规模 Android 应用保护机制实证研究 ^[18]	单点检测容易被绕过，运行时检测更适合作为风险评分信号

二、攻防实验与对比分析

2.1 实验设计与结果

本节选取两个本地 crackme 样例进行受控攻防验证。实验只在本地测试设备和模拟器中完成，不涉及真实业务系统。第一个样例是本地注册校验程序，目标是验证 Smali Patch 对客户端布尔判断的影响；第二个样例是注册码校验程序，目标是验证本地算法被还原后是否可以生成合法输入。实验使用 ApkTool、JADX/JD-GUI、Smali 编辑、keytool、apksigner 和 Java 编译环境。最终产物包括重打包签名后的测试 APK，以及用于第二个样例的 Keygen 程序。

公开移动安全训练中经常使用 crackme 或 UnCrackable Apps 作为逆向练习对象。OWASP MAS Crackmes 提供了 Android UnCrackable L1-L4、DRM 等练习，用于演示静态分析、动态调试、反调试绕过和代码篡改等技术^[7]。本文实验与这类训练目标一致：不是攻击真实应用，而是观察客户端授权逻辑为什么容易失效。

2.1.1 本地校验绕过

第一个实验从 APK 解包开始。使用 ApkTool 还原 Manifest、资源文件和 Smali 代码后，先确认主 Activity，再搜索界面提示字符串和校验相关关键词。定位到注册判断附近后，可以看到程序先调用校验函数，再根据返回值进入成功或失败分支。由于判断结果完全由客户端控制，只要修改条件跳转或强制返回成功，就可以改变后续执行路径。

修改完成后，使用 ApkTool 重新打包。未签名 APK 不能直接安装，因此继续使用测试证书签名，并在测试环境中安装运行。验证结果如图所示，程序进入解锁状态，说明本地校验已经被绕过。这个结果说明，客户端中的单点布尔判断不能承担最终授权功能；即使应用签名能够发现签名后的静默修改，攻击者仍可重新签名并诱导用户安装篡改版本。



图 4: 本地校验绕过后的解锁结果

2.1.2 注册码算法复现

第二个实验不直接修改 APK，而是分析注册码生成规则。使用 JADX/JD-GUI 查看反编译结果后，沿 validateSerial() 一类函数追踪输入注册码、设备标识、哈希计算和字符串截取之间的关系。确认校验逻辑后，用 Java 编写 Keygen 程序复现同样的数据处理过程，再把生成的注册码输入目标程序。

实验结果如图所示，程序显示“Code Accepted :D”和“Code is valid!”。这说明只要算法输入、计算过程和比较逻辑都保存在客户端，攻击者就可以在不修改 APK 的情况下复现授权规则。与 Patch 相比，注册机方式更隐蔽，因为应用包没有变化，单

纯的完整性校验不一定能发现问题。防护上应避免把长期授权算法完整放在客户端，关键状态应由服务端签发，并结合过期时间、随机 nonce、设备绑定和异常行为检测。

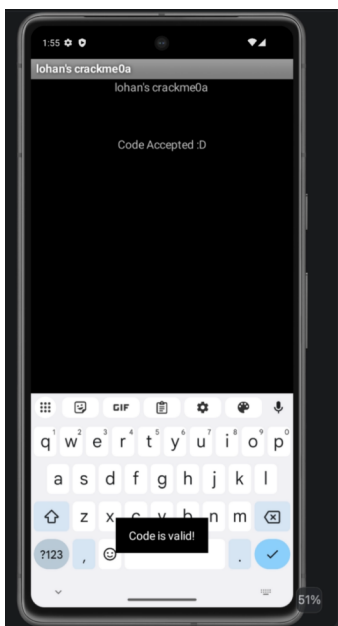


图 5: 注册码算法复现后的验证通过结果

2.1.3 实验结论

两个实验分别验证了客户端授权逻辑的两类风险：第一类是“分支可改”，即攻击者通过 Smali Patch 修改返回值或条件跳转；第二类是“算法可复现”，即攻击者还原本地注册码生成规则并编写注册机。二者共同说明，移动应用可以在客户端做体验层面的预校验，但不能让客户端成为最终可信判断者。真实应用应把会员、支付、积分、许可证等关键状态放到服务端校验，并把签名摘要、安装来源、版本状态和运行环境信号纳入风控判断^{[2][4]}。

2.2 实验分析与安全启示

在上述实验基础上，还可以把移动应用授权绕过归纳为三类路径：静态 Patch、算法复现和运行时 Hook。静态 Patch 直接修改安装包，攻击效果稳定，但会破坏原签名，容易留下重打包痕迹；算法复现不修改安装包，隐蔽性更强，但前提是校验算法和关键输入能从客户端还原；运行时 Hook 不改变 APK 文件，通过 Frida、Xposed 等工具在执行过程中替换函数返回值，适合快速验证安全假设，但对运行环境要求更高，也更容易触发反调试和反 Hook 检测。

我的观点是，防篡改不应只追求“让攻击者看不懂代码”，而应追求“即使攻击者看懂或局部修改，也难以获得真实业务收益”。例如，静态 Patch 可以让本地界面显

表 4: 三类授权绕过方式的对比

绕过方式	攻击特点	风险表现	主要防护思路
静态 Patch	修改 Smali、常量或跳转逻辑，重打包后重新签名	能稳定改变客户端逻辑，但会改变 APK 文件和签名状态	签名摘要校验、DEX/so 完整性校验、安装来源判断、服务端拒绝异常签名客户端
算法复现	还原注册码或授权算法，编写外部 Keygen	不一定修改 APK，普通完整性校验难以发现	服务端签发授权、随机挑战、短期 token、密钥轮换、避免长期秘密写入客户端
运行时 Hook	在运行时替换函数返回值或拦截加解密过程	适合绕过反调试、证书绑定、授权判断等动态逻辑	反 Hook 检测、环境风险评估、关键逻辑 native 化、多点检测与服务端风控联动

示成功，但如果服务端不承认该客户端签名和授权状态，攻击就只能停留在假界面；算法复现可以生成本地注册码，但如果授权结果需要服务端签名并定期刷新，注册机就无法长期有效；运行时 Hook 可以改函数返回值，但如果服务端同时观察设备环境、请求频率、nonce 重放和版本状态，攻击者就很难低成本规模化使用。基于这一点，本文后文提出的“客户端可信度评分”并不是把某个检测点当成绝对依据，而是把签名、完整性、环境、行为和服务端状态组合起来，形成分级处置机制。这种思路比单纯加壳更适合真实移动应用。

2.2.1 关键代码与安全启示

结合 crackme 类程序的实验过程，可以把一次典型注册校验定位抽象成如下逻辑。界面层接收用户输入后调用 check_code() 或 validateSerial()，再根据返回值决定跳转到成功分支还是失败分支。逆向时最有效的做法不是一开始就阅读所有代码，而是先搜索界面提示字符串，找到“注册成功”“注册失败”所在方法，再向上追踪条件跳转和返回值来源。

Listing 1: 注册校验逻辑的 Java 伪代码

```

1 boolean ok = check_code(input);
2 if (ok) {
3     showMessage("注册成功");
4     enableVipFeature();
5 } else {
6     showMessage("注册失败");
7 }

```

在 Smali 层，这种逻辑通常会表现为一次方法调用、一次 move-result 取返回值，以及一次条件跳转。假设 v0 保存 check_code() 的返回值，if-eqz v0, :cond_fail 表示“如果返回值为 0，就进入失败分支”。因此攻击者只要把返回值强制改成 1，或把失

败跳转改成成功跳转，就能绕过本地校验。

Listing 2: Smali 层校验分支的典型形态

```
1  invoke-virtual {p0, v1}, Lcom/example/MainActivity;->check_code(Ljava/lang/  
    String;)Z  
2  move-result v0  
3  if-eqz v0, :cond_fail  
4  
5  :cond_success  
6  const-string v2, "注册成功"  
7  invoke-virtual {p0, v2}, Lcom/example/MainActivity;->showMessage(Ljava/lang/  
    String;)V  
8  return-void  
9  
10 :cond_fail  
11 const-string v2, "注册失败"  
12 invoke-virtual {p0, v2}, Lcom/example/MainActivity;->showMessage(Ljava/lang/  
    String;)V  
13 return-void
```

如果采用最直接的 Patch，可以在 move-result 后插入 const/4 v0, 0x1，使后续分支永远认为校验通过。这个修改非常小，但破坏力很大，因为它绕过的不是某个界面提示，而是整个授权判断的根节点。

Listing 3: 强制返回成功的一种 Patch 思路

```
1  invoke-virtual {p0, v1}, Lcom/example/MainActivity;->check_code(Ljava/lang/  
    String;)Z  
2  move-result v0  
3  const/4 v0, 0x1 # 强制把校验结果改为 true  
4  if-eqz v0, :cond_fail # 该分支不再会被触发
```

注册机开发的思路则不同。它并不修改应用，而是还原 validateSerial() 的算法。若注册码由“设备标识 + 固定盐值 + MD5 + 截取/大小写转换”生成，那么攻击者可以写出等价程序。下面的伪代码展示了这一类算法的风险：只要设备标识和盐值可被逆向获得，注册码就不再是秘密。

Listing 4: 本地注册码算法的风险示例

```
1  String deviceId = getDeviceId();  
2  String raw = deviceId + ":fixed_salt";  
3  String md5 = md5(raw).toUpperCase();  
4  String serial = md5.substring(0, 8) + "-" + md5.substring(8, 16);  
5  return input.equals(serial);
```

这段程序分析说明，客户端授权的核心问题并不是“攻击者是否知道按钮在哪里”，而是“授权判断是否完全由客户端掌握”。只要最终判断点、算法输入和比较结果

都在本地，攻击者就能在静态 Patch、动态 Hook 和算法复现之间选择成本最低的一条路。

2.2.2 实验暴露的安全问题

第一，客户端本地验证不应承担最终授权。crackme 类程序的核心问题是把“是否注册成功”的最终判断放在本地。只要攻击者能修改本地分支或复现本地算法，就能绕过授权。真实商业应用中，会员状态、支付状态、许可证有效期和敏感功能开关应由服务端签发并校验。客户端可以缓存短期 token，但 token 应带有签名、过期时间、设备绑定和可吊销机制。

第二，设备标识不是安全密钥。很多早期移动应用会使用 IMEI、MAC、Android ID 等设备信息作为注册码输入。问题在于，这些标识可能为空、不可访问、被系统限制、被模拟器伪造或被 Hook 修改。现代 Android 对设备标识访问越来越严格，隐私合规也不允许随意读取敏感标识。因此，授权绑定应转向更合规的设备指纹、账号体系、服务端风险模型和硬件安全能力，而不是直接依赖单一设备 ID。

第三，静态字符串会显著降低逆向成本。实验中通过搜索“注册成功”“register”“emulator”等关键词就能快速定位关键逻辑。真实应用应尽量避免把关键提示、接口路径、密钥片段和安全策略以明文形式集中暴露。字符串加密并不能彻底防止逆向，但能提高攻击者定位成本，尤其适合保护关键分支附近的信息。

第四，反模拟器和反调试只能作为辅助。实验里模拟器检测可以被直接修改 Smali 绕过，这说明单点检测很脆弱。更稳妥的方法是把环境检测作为风险信号，而不是唯一阻断条件。例如应用可以把检测结果上报服务端，由服务端结合账号行为、设备画像、签名摘要和接口异常进行综合判断。

第五，签名校验必须和来源治理结合。攻击者重打包后必须重新签名，系统层面会阻止它直接覆盖原签名应用，但用户仍可能卸载正版后安装篡改版。开发者可以在应用启动和关键接口中校验自身签名，并在服务端拒绝非官方签名的客户端请求。同时，应用市场应加强对相似包名、仿冒图标、恶意重打包和盗版分发的审核。

2.3 防护思路对比与改进设想

Android 和 iOS 的应用逆向风险差异，根源在生态模式不同。Android 更开放，允许多渠道分发和侧载，应用包更容易获取，Java/Kotlin 字节码也相对容易反编译。因此 Android 应用更依赖混淆、加固、签名校验、应用市场治理和服务端风控。iOS 生态更封闭，应用主要通过 App Store 分发，系统对侧载和调试限制更强，应用二进制以 Mach-O 原生代码为主，普通用户获取和安装篡改版应用的门槛较高。但 iOS 并非

没有逆向风险。越狱设备、企业证书滥用、动态库注入、Frida/LLDB 调试、Mach-O 静态分析、Objective-C runtime 特性都可能被攻击者利用。

从防护目标看，Android 更需要面对“重打包分发”风险，iOS 更常面对“越狱环境下的运行时 Hook”和“企业签名灰色分发”风险。Android 的 APK 签名、Play Protect、应用市场审核和系统权限模型是基础防线，但第三方市场质量不一会削弱整体效果。iOS 的 App Store 审核和系统封闭性提供了更强默认保护，但一旦用户进入越狱或灰色签名环境，应用同样需要自校验、反调试和服务端风控。

从开发者角度看，Android 防护更强调“默认不信任客户端”。因为 APK 容易被反编译和修改，所以关键逻辑应尽量服务端化；必须放在客户端的逻辑，应通过混淆、native 下沉、完整性校验、加固和风控降低风险。iOS 开发者也不能因为生态封闭就忽略客户端安全，尤其是金融、游戏、音视频版权和企业数据类应用，需要对越狱、注入、调试和敏感数据存储进行防护。

表 5: Android、iOS 与 HarmonyOS/OpenHarmony 在应用防篡改方面的差异

维度	Android	iOS	鸿蒙/开源鸿蒙
分发模式	官方市场与第三方市场并存，侧载较常见	App Store 为主，侧载和企业签名受限制	应用市场与生态迁移并行，逐步形成 HAP、APP Pack 分发体系
包格式	APK/AAB, DEX 字节码与 so 库并存	IPA, Mach-O 原生二进制为主	HAP/HSP/APP Pack, Ability 模型和 ArkTS/ArkUI 生态
主要风险	重打包、仿冒分发、Hook、恶意 SDK	越狱注入、动态调试、企业证书滥用	生态成熟期的工具链不足、跨设备攻击面、兼容迁移风险
防护重点	签名校验、混淆加固、服务端风控、应用市场治理	越狱检测、代码签名、运行时保护、服务端风控	签名/Profile、Access Token、appverify、分布式授权和开发者安全工具

2.3.1 防护方案分层评估

为了更系统地评价移动应用防篡改技术，可以把它分为五层：代码层、包完整性层、运行环境层、业务协议层和生态治理层。

代码层关注源代码和二进制是否容易理解。混淆、字符串加密、控制流混淆、native 下沉、虚拟机保护都属于这一层。这一层的价值是提高理解成本，理由是攻击者定位关键分支、密钥和接口需要花费更多时间；但它无法保证攻击者完全无法分析，尤其在动态 Hook 面前，代码层保护容易被绕开。

包完整性层关注安装包是否被改动。Android v2/v3 签名方案对 APK 内容摘要进行保护，系统安装时可以发现篡改。应用自身也可以校验签名证书、DEX 哈希和 so 哈希。这一层的价值是识别重打包，理由是篡改安装包通常会破坏原始签名和文件摘要；但对“攻击者重新签名后诱导用户安装”的场景，还需要结合服务端验证。



图 6: 移动应用防篡改的分层防护模型

运行环境层关注应用是否运行在被调试、Hook、Root、模拟器或注入环境中。反调试、反 Hook、Root 检测、模拟器检测、SELinux 状态检查、加载库扫描都属于这一层。这一层的价值是发现高风险环境，理由是动态分析和运行时篡改通常需要异常调试、注入或系统改动条件；但误报和绕过都较常见，因此更适合作为风险评分输入。

业务协议层关注客户端与服务端之间的信任关系。接口签名、时间戳、nonce、防重放、证书绑定、服务端授权、设备绑定、行为风控、密钥轮换都属于这一层。这一层通常是最关键的，因为攻击者即使控制客户端，也未必能伪造服务端认可的状态。真正成熟的移动安全体系，一定会把关键授权放在业务协议层闭环。

生态治理层关注分发渠道和用户侧风险。官方市场审核、仿冒应用下架、恶意域名拦截、证书信誉、漏洞响应、用户提示、隐私合规和供应链审计都属于这一层。移动应用不是单独存在的文件，而是处在应用市场、操作系统、云服务和用户设备共同构成的生态中。没有生态治理，单个应用的技术防护很难覆盖所有传播风险。

2.3.2 个人观点与改进设想

我认为移动应用防篡改最容易出现的误区，是把它当成“逆向工程师和加固壳之间的技术比赛”。这种理解只看到了局部。真正的目标不是让应用永远无法被逆向，因为在攻击者拥有设备和安装包的情况下，这几乎不现实。更合理的目标是：让攻击者无法低成本、稳定、规模化地获得收益。只要篡改成本高于收益，或者篡改版本无法通过服务端验证，防护就已经发挥了实际价值。

从个人使用体验看，移动应用安全并不是离用户很远的底层问题。平时安装应用时，我会明显感受到不同来源带来的安全差异：官方应用市场通常会给出版本、权限和隐私说明，而从网盘、论坛或聊天群下载的 APK 往往缺少来源证明，用户很难判断它是否被重打包。使用支付、网盘、校园服务和位置类应用时，系统弹出的相机、定位、通讯录、存储等权限请求也会直接影响信任感。如果一个应用在不相关场景中申请过多权限，即使它没有被逆向篡改，也会让用户怀疑其数据处理是否合理。

由此看，移动安全不仅是开发者对抗攻击者的问题，也是系统、应用市场和用户共同建立信任的问题。

基于这个思路，可以设计一种“客户端可信度评分”机制。客户端不再简单地因为发现一个风险点就立即退出，而是把签名摘要、安装来源、代码完整性、Hook 痕迹、调试状态、设备环境、接口调用节奏、账号历史行为、地理位置异常、版本更新状态等信号汇总成风险分。低风险用户正常使用；中风险用户触发二次验证、限制高价值操作或要求更新；高风险客户端拒绝关键交易并上报。这种做法比单点反调试更灵活，也能减少误伤。

这种机制的理由在于，移动终端处在用户可控制的环境中，任何单一检测都可能出现误报或被绕过。把多个弱信号组合起来，可以让服务端根据风险程度采取差异化措施。例如 Root 痕迹本身不一定代表攻击，但如果同时出现非官方签名、代码哈希异常、Frida 进程痕迹和高频接口调用，就更可能是篡改客户端。下面给出一个简化伪代码，用于说明评分思想。

Listing 5: 客户端可信度评分的简化伪代码

```
1  score = 100
2
3  if signatureDigest != officialDigest:
4      score -= 35
5  if dexHashChanged or nativeLibHashChanged:
6      score -= 25
7  if installSource not in trustedStores:
8      score -= 15
9  if debuggerDetected or hookFrameworkDetected:
10     score -= 20
11 if emulatorDetected or rootDetected:
12     score -= 10
13 if requestRateAbnormal or replayNonceDetected:
14     score -= 20
15
16 if score >= 80:
17     allowNormalService()
18 elif score >= 50:
19     requireStepUpVerification()
20 else:
21     blockHighValueOperation()
22     reportRiskEvent()
```

另一个改进方向是“安全策略远程化”。很多应用把检测规则写死在客户端，例如检测某个 Frida 字符串或某个进程名。攻击者一旦知道规则，就能针对性绕过。若应用把部分检测策略、哈希基准、风险阈值和挑战任务由服务端动态下发，攻击者就需要持续跟进变化。这里要注意，远程策略本身也要签名和防重放，避免被中间人

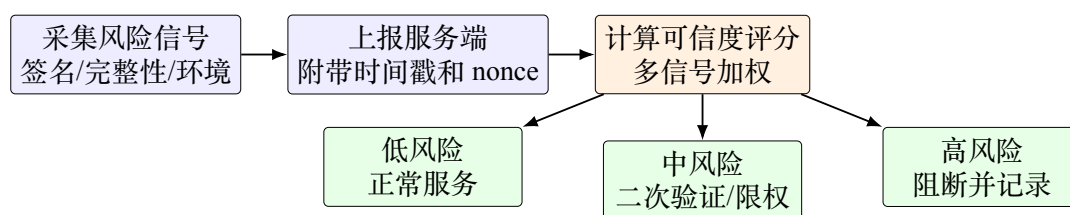


图 7: 客户端可信度评分与分级处置流程

篡改。

第三个方向是“把逆向分析结论转化为开发检查清单”。crackme 类样例虽然结构简单，但它暴露的问题在真实应用中同样存在。每写一个注册、会员、支付、积分或权限判断功能，都可以反问：如果攻击者把这个函数返回值改成 true 会怎样？如果攻击者复现这个算法会怎样？如果攻击者抓包重放这个接口会怎样？如果攻击者用非官方签名客户端请求服务端会怎样？这些问题可以变成开发阶段的安全评审表，而不是等上线后再靠加固补救。

第四个方向是“国产系统和 Android 安全经验互相借鉴”。HarmonyOS/OpenHarmony 在应用模型、权限、分布式设备和应用包格式上与 Android 不完全相同，但逆向与防篡改的核心矛盾仍然存在：客户端不可信、安装包可被分析、运行时可被观察、权限容易被滥用、分发渠道需要治理。因此，国产系统不应只强调“新格式、新语言、新生态”，还应吸收 Android 多年攻防经验，在应用签名、完整性校验、权限最小化、应用市场审核、开发者安全工具和服务端风控之间形成完整闭环。

三、鸿蒙等国产系统在该领域的发展现状

3.1 国产移动操作系统与应用安全背景

国产移动操作系统的发展与移动安全密切相关。过去很长一段时间，国内智能终端生态主要依赖 Android 及其深度定制系统。厂商可以基于 Android 做界面、驱动、系统服务和应用市场优化，但底层架构、应用兼容、系统升级和安全机制仍然受到 Android 生态影响。随着移动终端成为数字经济和关键业务入口，操作系统自主可控、应用生态安全、隐私保护和供应链可信的重要性不断上升。HarmonyOS、OpenHarmony、统信、麒麟等国产系统在不同终端形态上发展，体现了国内在操作系统领域从“适配和定制”向“自研生态和基础能力建设”推进的趋势。

HarmonyOS 的特点是面向多设备、全场景和分布式协同。官方开发者资料强调，HarmonyOS 应用开发使用 DevEco Studio、HarmonyOS SDK、ArkTS、ArkUI 等工具和框架，面向手机、平板、穿戴、车机、智慧屏等设备提供一次开发、多端部署能

力^[8]。OpenHarmony 则是开放原子开源基金会孵化的开源项目，面向更广泛的设备厂商和行业场景。对移动应用安全而言，这种全场景架构既带来机会，也带来挑战。机会在于系统可以从应用模型、权限、签名、分布式身份和设备可信关系上重新设计安全边界；挑战在于跨设备流转、应用接续、多端部署和元服务会扩大攻击面，需要比传统手机 App 更细粒度的权限和完整性治理。

从近年的发展看，HarmonyOS NEXT 的发布使国产移动系统从“兼容和迁移”进一步走向“原生生态建设”。华为官网新闻资料在介绍 HarmonyOS NEXT 时强调原生精致、原生互联、原生智能、原生安全、原生流畅等方向^[13]。这与本文讨论的防篡改技术直接相关：当系统逐步形成原生 HAP/APP Pack 应用生态后，应用签名、安装校验、权限模型、应用市场审核和开发者工具链就会成为国产系统安全能力能否落地的关键。换句话说，HarmonyOS/OpenHarmony 的安全竞争力不只取决于内核或某一个安全模块，也取决于它能否让大量普通开发者在日常开发中默认采用安全做法。

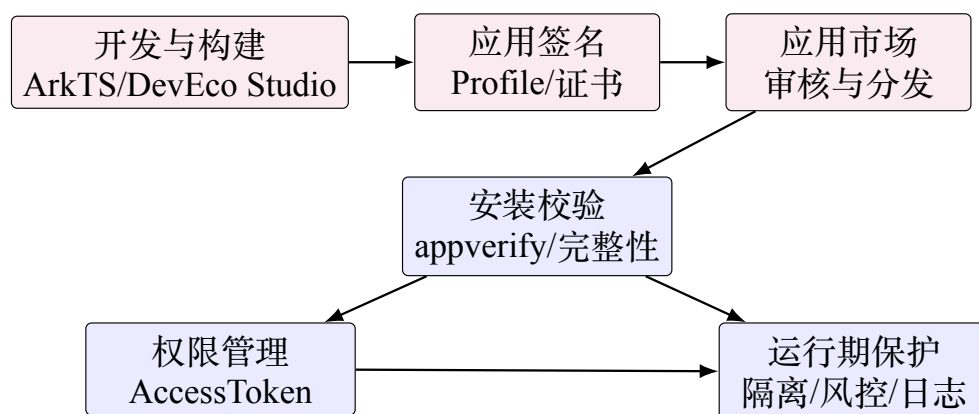


图 8: HarmonyOS/OpenHarmony 应用安全链路示意

3.2 HarmonyOS/OpenHarmony 的应用包与签名体系

在 Android 中，应用包主要是 APK/AAB；在 HarmonyOS/OpenHarmony 生态中，应用包通常涉及 HAP、HSP、APP Pack 等概念。HAP 是 HarmonyOS Ability Package，用于承载应用模块代码、资源和配置；APP Pack 可以包含一个或多个 HAP/HSP，用于发布和分发。由于应用包格式、Ability 模型、ArkTS/ArkUI 和方舟运行时与 Android APK 不完全相同，传统 Android 逆向工具不能完全照搬，但“包结构分析、字节码/中间表示分析、资源分析、签名验证和运行时 Hook”的安全问题依然存在。

应用签名仍是国产系统应用安全的关键基础。华为开发者文档中有 DevEco Studio 签名配置相关指南，OpenHarmony 的 security_appverify 仓库也说明，应用完整性验证模块通过签名验证确保待安装应用的完整性和可信来源，并在安装阶段由应用框架子系统调用相关验证接口^[12]。这说明应用安装前的来源可信和内容完整性是系

统安全要求的一部分。与 Android 类似，签名可以帮助系统确认应用开发者身份、阻止未授权修改直接通过安装校验，并与权限等级、ACL 和系统应用身份关联。

从防篡改角度看，HarmonyOS/OpenHarmony 的签名体系有两个特点：一是签名不只决定“能不能安装”，还与权限等级、系统应用身份、调试设备、发布证书和 Profile 文件相关；二是应用市场和设备侧安装校验可以形成双重治理，分别在发布前、安装时和运行时限制风险。

3.3 AccessToken 权限模型与应用隔离

OpenHarmony 的 AccessTokenManager 是应用权限管理的重要机制。相关官方/开源文档说明，TokenID 在系统中唯一标识应用，AccessTokenManager 基于 TokenID 管理应用的 Access Token 信息，包括应用 ID、子用户 ID、APL 等级和权限授权状态；当应用访问资源时，系统根据 TokenID 获取权限授权状态并进行鉴权^[11]。这种设计与 Android 基于 UID、权限和 AppOps 的模型有相似目标：把应用身份、权限状态和资源访问控制关联起来，避免应用越权访问敏感数据或系统能力。

对移动应用逆向和防篡改而言，权限模型有两层意义。第一，权限最小化可以降低篡改版应用的危害。如果一个普通应用即使被重打包，也无法获得联系人、定位、麦克风、文件、设备管理等敏感权限，那么攻击者收益会下降。第二，权限等级和签名绑定可以保护系统能力。OpenHarmony 权限体系中存在不同开放范围和授权方式，一些受限权限需要 ACL、系统签名或更高 APL 等级。这样可以防止普通应用通过简单声明权限就获取系统级能力。

不过，权限模型并不能替代应用自身安全。攻击者可以篡改应用逻辑，在应用已经拥有的权限范围内滥用数据。例如一个合法金融应用本来就有网络访问、相机扫码和本地存储权限，如果被植入恶意逻辑，仍可能上传敏感信息。因此，权限控制解决的是“系统是否允许访问资源”，防篡改解决的是“应用代码是否仍是开发者预期的代码”，两者需要配合。

3.4 国产系统在该领域的优势

第一，国产系统有机会在生态建设阶段直接把安全、隐私和权限最小化纳入开发规范。Android 生态历史包袱较重，很多机制需要在兼容性和安全之间折中；HarmonyOS/OpenHarmony 则可以更早把应用隐私保护、数据安全、安全编码、资产保护和代码混淆纳入开发者质量指导^{[8][9][10]}。

第二，多设备统一生态有利于建立端到端信任。HarmonyOS 强调全场景、多设备协同，如果能把设备身份、应用签名、用户授权、分布式数据访问和可信执行环境

统一起来，跨设备流转就能形成更完整的安全链。

第三，应用市场和开发者工具可以更集中地推动安全规范。HarmonyOS 应用上架前需要做漏洞、隐私、兼容性、稳定性和性能测试；如果 DevEco Studio、云测试和专项测试进一步集成混淆检查、敏感权限检查、明文密钥扫描和签名配置检查，效果会更直接。

第四，国产系统可以结合国内合规要求强化隐私和数据安全。移动应用安全不仅是技术问题，也涉及个人信息保护、数据出境、最小必要、告知同意和敏感权限使用；把这些要求和权限弹窗、应用市场审核、系统 API 限制结合起来，能减少过度索权和违规 SDK 风险。

第五，软硬件协同空间更大。移动终端安全离不开硬件根信任、安全启动、TEE、密钥库和系统完整性保护；如果国产系统能与国产芯片、安全硬件和密钥管理协同设计，就能为应用签名校验、密钥保护、设备认证和支付安全提供更强底座。

3.5 国产系统在该领域的不足与挑战

第一，生态成熟度仍需时间。移动应用安全工具链的成熟离不开真实攻击样本、漏洞响应、开发者实践和第三方安全公司参与。Android 已形成 JADX、Frida、OWASP MASTG、MobSF 等较完整的生态，而 HarmonyOS/OpenHarmony 面向 HAP、ArkTS、方舟字节码和分布式场景的分析与测试工具还需要继续丰富。

第二，跨设备场景扩大了攻击面。传统 Android 应用主要在单个手机上运行，而 HarmonyOS 强调应用接续、多端部署和分布式协同，安全边界也从单机应用扩展为多个设备和服务之间的协同，这会带来新的授权和撤销信任问题。

第三，兼容与安全之间仍会冲突。为了吸引开发者和用户，国产系统需要较好的迁移体验，但兼容旧生态也可能继承重打包、过度权限和恶意 SDK 等问题。

第四，开发者安全能力参差不齐。即使系统提供签名、权限和完整性校验，开发者仍可能把密钥写进代码、把授权逻辑放在客户端或忽略服务端校验。移动安全最终还是落到每个应用的工程实践上。

第五，第三方 SDK 和供应链治理压力较大。现代移动应用往往集成广告、统计、推送、支付、地图、客服、登录和风控等大量 SDK，任何一个 SDK 的漏洞、过度采集或恶意更新都可能影响应用安全。

3.6 建议与展望

第一，建立面向 HarmonyOS/OpenHarmony 的移动应用安全验证标准。可参考 OWASP MASVS 的分层思想，把应用安全要求划分为数据存储、密码、认证、网络、平台交互、代码质量、抗逆向、防篡改、隐私和分布式安全等控制组。

第二，在 DevEco Studio 和构建流程中集成安全检查。建议在编译发布时自动检查签名配置、敏感权限、明文密钥、硬编码 URL、调试开关、日志泄露和第三方 SDK 版本，把安全尽量前移到开发阶段。

第三，加强 HAP/HSP/APP Pack 的完整性和来源校验透明度。系统和应用市场可以提供更清晰的签名证书、安装来源、版本链路和更新记录，并对篡改版和非官方签名版本给出更强提示或拦截。

第四，完善跨设备安全策略。建议对跨设备剪贴板、文件流转、应用接续、远程调用和设备间认证等场景建立细粒度权限、可撤销授权和审计日志，高敏感业务则应按设备安全等级动态限制能力。

第五，推动国产安全工具链和社区建设。OpenHarmony 作为开源项目，可以鼓励社区建设 HAP 分析器、ArkTS 审计工具、权限可视化工具、签名检查工具和漏洞复现实验环境，让安全研究更容易开展。

第六，强调服务端和终端联动。无论 Android 还是 HarmonyOS，客户端都不能被完全信任。国产系统可以提供更标准化的设备可信证明、应用完整性证明、密钥保护和风险信号接口，让应用服务端能够判断请求是否来自可信设备、可信应用和可信运行环境。这样可以把系统安全能力转化为业务安全能力。

总体来看，HarmonyOS/OpenHarmony 在移动应用安全领域已经具备较好的基础：有应用包签名与安装校验，有 AccessToken 权限管理，有安全子系统和开发者安全指导，也有应用市场发布测试流程。它的优势在于新生态可以更系统地吸收安全经验，并把多设备协同、国产芯片和开发工具链结合起来；不足在于生态仍需积累，工具链、社区、第三方 SDK 治理和跨端安全规范还需要继续完善。未来国产系统如果把“系统级可信、应用级防篡改、服务端风控、市场治理、开发者工具”真正串成闭环，就有机会在移动应用安全领域形成区别于传统 Android 定制系统的新优势。

四、结论

移动应用逆向分析与防篡改技术是移动终端安全中的典型攻防领域。它与教材和课程中涉及的开放式移动操作系统、Android 安全模型、应用签名、权限控制、Linux/ARM 底层安全和恶意应用风险都有直接关系^[1]。通过对 APK 结构、DEX/Smali、

签名机制、混淆加固、反调试、反 Hook 和服务端风控的分析可以看到，移动应用安全的核心矛盾在于：应用必须在用户可控制的终端上运行，但开发者又希望保护代码、数据和业务逻辑不被篡改。

crackme 类实验说明，单纯依赖客户端判断的注册、授权和设备绑定机制很容易被静态 Patch 或算法复现绕过。真正可靠的方案应当分层构建：代码层提高理解成本，签名和完整性层识别重打包，运行环境层发现调试和 Hook，业务协议层由服务端完成最终授权，生态治理层减少仿冒和恶意分发。防护的目标不是绝对阻止逆向，而是让攻击者难以低成本、稳定、规模化获利。

国产系统方面，HarmonyOS/OpenHarmony 在应用包、签名、权限、应用市场、开发工具和分布式安全方面正在形成自己的体系。其优势是可以在新生态中更早引入安全规范，并结合多设备、国产芯片和开发者工具链建立端到端信任；不足是生态成熟度、第三方安全工具、跨设备攻击面治理和开发者安全实践仍需加强。未来移动应用安全的发展方向，应当是系统、应用、服务端和应用市场共同参与的全生命周期安全，而不是某一个加固工具或某一条检测规则的孤立防护。

参考文献

- [1] 张森:《移动终端安全》,北京邮电大学出版社,2019,ISBN: 9787563558858。
- [2] Android Open Source Project. APK signature scheme v2. <https://source.android.com/docs/security/features/apksigning/v2>
- [3] Android Open Source Project. APK signature scheme v3. <https://source.android.com/docs/security/features/apksigning/v3>
- [4] Android Developers. apksigner. <https://developer.android.com/tools/apksigner>
- [5] OWASP. MASVS-RESILIENCE: Resilience Against Reverse Engineering and Tampering. <https://mas.owasp.org/MASVS/11-MASVS-RESILIENCE/>
- [6] OWASP. M9: Reverse Engineering. <https://owasp.org/www-project-mobile-top-10/2016-risks/m9-reverse-engineering>
- [7] OWASP. Android Crackmes. <https://mas.owasp.org/crackmes/Android/>
- [8] 华为开发者联盟: HarmonyOS 应用开发知识地图。 <https://developer.huawei.com/consumer/cn/app/knowledge-map/>
- [9] 华为开发者联盟: 应用资产保护设计, 见 HarmonyOS 应用开发知识地图“应用质量/隐私与安全”。 <https://developer.huawei.com/consumer/cn/app/knowledge-map/>
- [10] 华为开发者联盟: 应用代码混淆, 见 HarmonyOS 应用开发知识地图“应用质量/隐私与安全”。 <https://developer.huawei.com/consumer/cn/app/knowledge-map/>
- [11] OpenHarmony Docs. Application Permission Management Overview. <https://github.com/openharmony-rs/openharmony-docs/blob/master/en/application-dev/security/AccessToken/app-permission-mgmt-overview.md>
- [12] OpenHarmony. security_appverify: Application integrity verification module. https://github.com/openharmony/security_appverify
- [13] 华为: HarmonyOS NEXT 正式发布, 开启更多机型公测。 <https://www.huawei.com/cn/news/2024/10/harmonyos5>

- [14] William Enck, Damien Ocateau, Patrick McDaniel, Swarat Chaudhuri. A Study of Android Application Security. USENIX Security Symposium, 2011. <https://www.usenix.org/conference/usenixsecurity11/study-android-application-security>
- [15] Wu Zhou, Yajin Zhou, Xuxian Jiang, Peng Ning. Detecting Repackaged Smartphone Applications in Third-Party Android Marketplaces. CODASPY, 2012. <https://www.sciweavers.org/publications/detecting-repackaged-smartphone-applications-third-party-android-marketplaces>
- [16] Parvez Faruki, Hossein Fereidooni, Vijay Laxmi, Mauro Conti, Manoj Singh Gaur. Android Code Protection via Obfuscation Techniques: Past, Present and Future Directions. arXiv, 2016. <https://arxiv.org/abs/1611.10231>
- [17] Vivek Balachandran, Sufatrio, D. J. J. Tan, V. L. L. Thing. Control flow obfuscation for Android applications. Computers & Security, 2016. DOI: <https://doi.org/10.1016/j.cose.2016.05.003>
- [18] Stefano Berlato, Mariano Ceccato. A large-scale study on the adoption of anti-debugging and anti-tampering protections in Android apps. Journal of Information Security and Applications, 2020. DOI: <https://doi.org/10.1016/j.jisa.2020.102463>

致谢

感谢张森老师在“移动终端安全”课程中的讲授。课程围绕移动终端技术基础、Android 操作系统原理、移动系统安全机制、安全漏洞验证、软件破解分析、应用加固与防护等内容展开，使我能够从系统、应用、攻击与防护多个角度理解移动终端安全问题。

张森老师的课程内容和课件为我理解本文选题提供了重要帮助。课程中关于开放式移动操作系统、应用签名、权限控制、APK 结构、逆向分析流程和软件保护方法的讲解，使我认识到移动应用安全并不只是单个工具的使用，而是涉及终端环境、应用逻辑、系统机制、分发渠道和服务端风控的综合问题。

通过本课程的学习，我对移动终端安全的体系化理解更加清晰，也更加认识到安全分析应当服务于合规研究和防护改进。谨向张森老师表示诚挚感谢。